Lecture 9

Decoding

Rats in Space! on the DANDI Archive

Hippocampal Recordings from the Space Shuttle

In 1998, three rats flew aboard the Space Shuttle as part of the Neurolab mission, carrying tiny electrode arrays in their brains. The goal: to find out whether the hippocampus, the brain's internal GPS, can build a map of space when there is no gravity. Over 25 years later, these historic recordings are now publicly available on DANDI as Dandiset #001754.

https://about.dandiarchive.org/blog/2026/02/27/place-cells-in-space-hippocampal-recordings-from-the-space-shuttle/

What is decoding?

Decoding is the process of **inferring the underlying representation** from observed neural activity, typically spike times (or continuous data like LFPs or EEG).

Given some neural signal (e.g., spike train), the goal is to **predict a cognitive or behavioral variable (like position, decision, or sensory stimulus)**.

Note: this could be something continuous (like position) or discrete (like decision left or right)

Contrast this to encoding (first set of lectures)

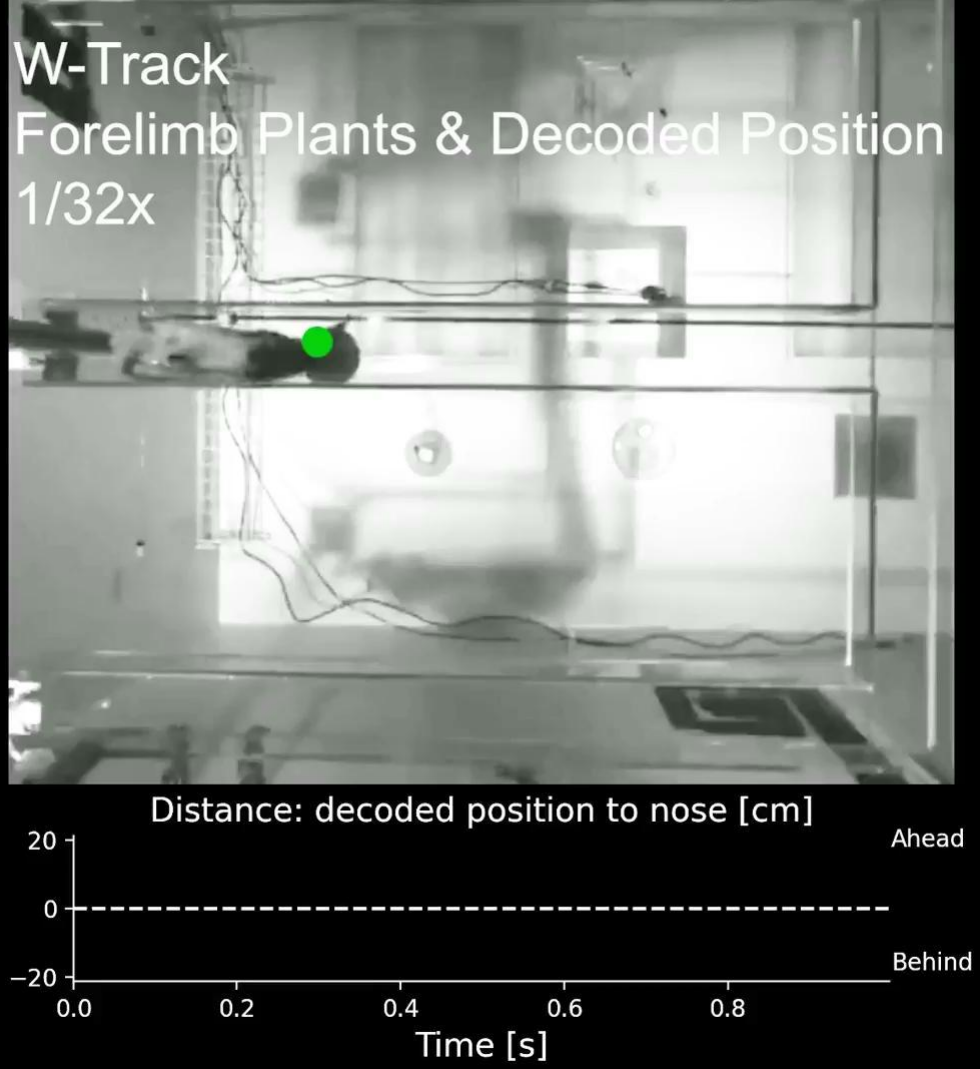
This asked the question: **given external stimuli or behavior** (e.g., sensory input, task parameters, movement), **what is the spiking of a neuron?**

Question: *“How do the firing rates of place cells change as the rat moves through different locations in the maze?"*

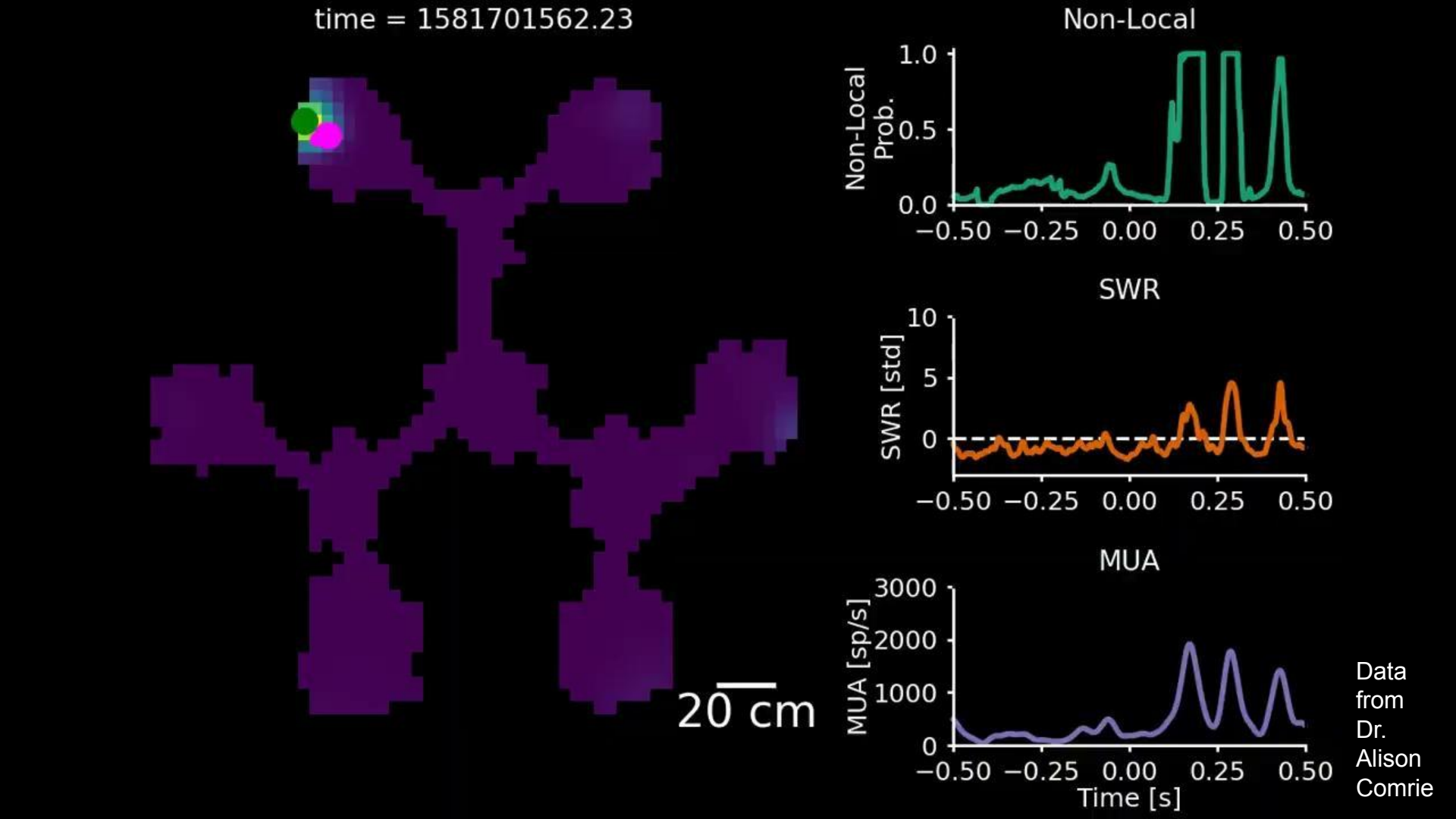
How is decoding used in spatial neuroscience?

Classic example: Hippocampus

Question: *“Given spike times from hippocampal neurons, where is the rat in the maze?”*

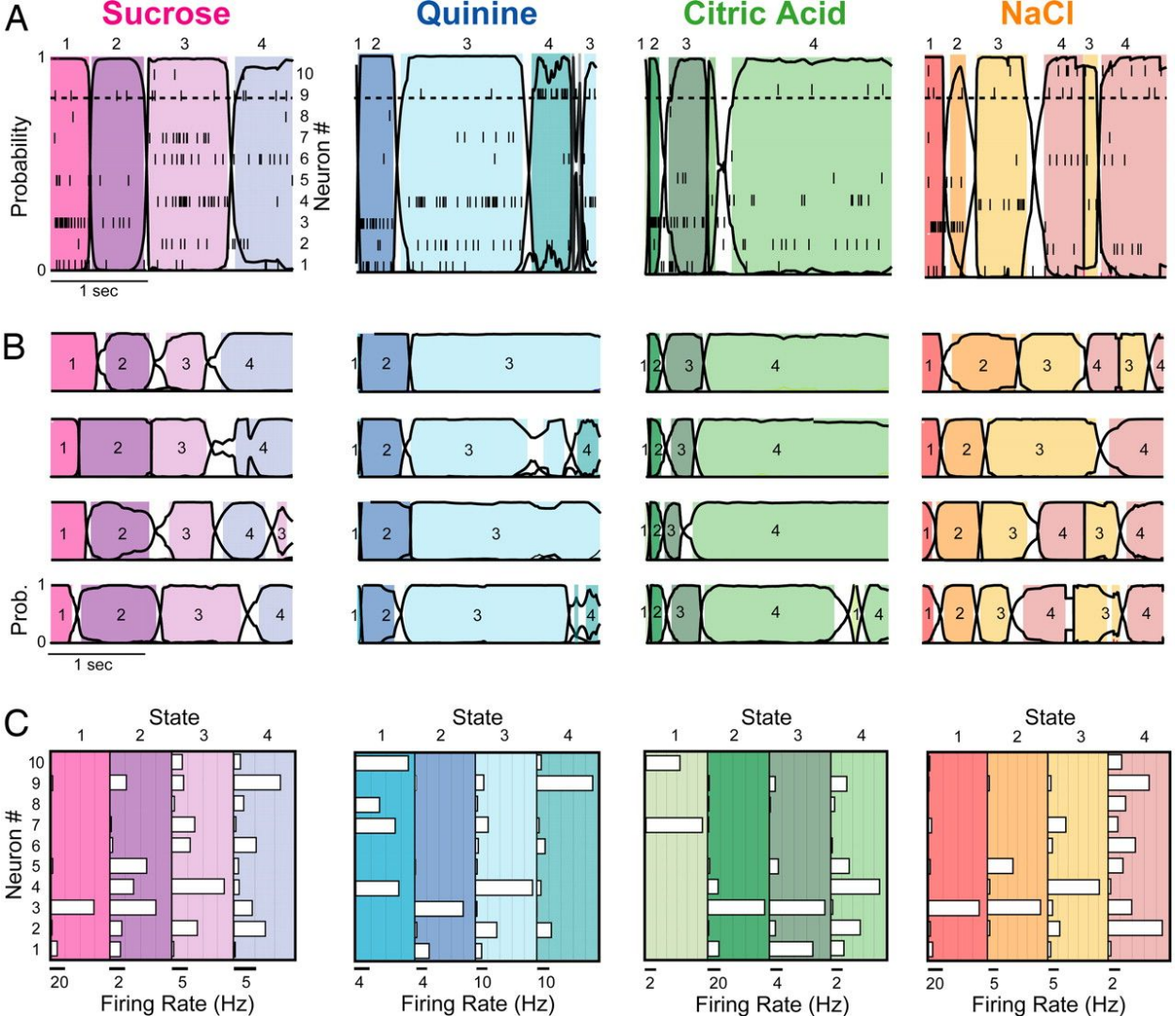


Joshi et al. 2023



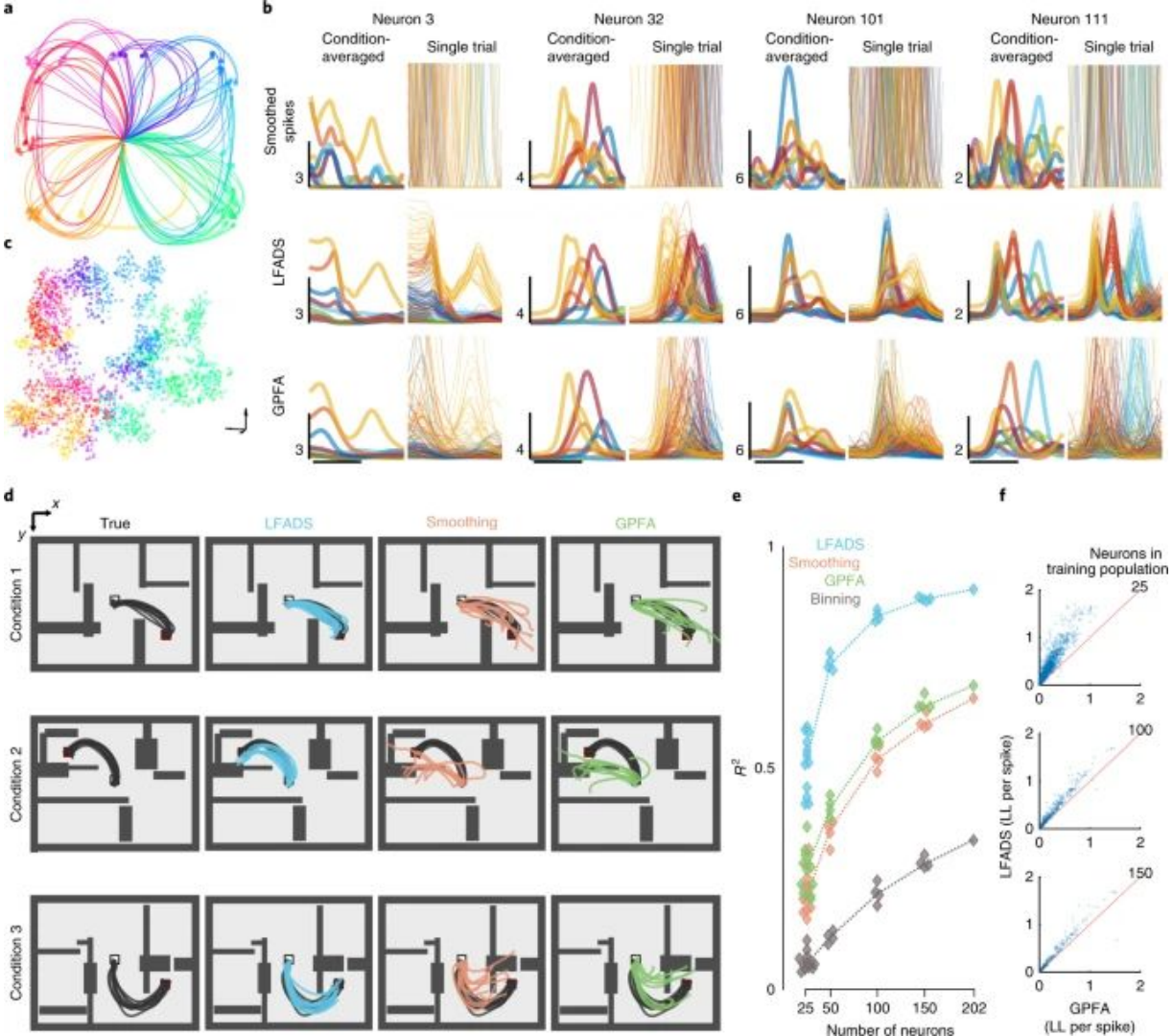
Data from Dr. Alison Comrie

Sensory Decoding



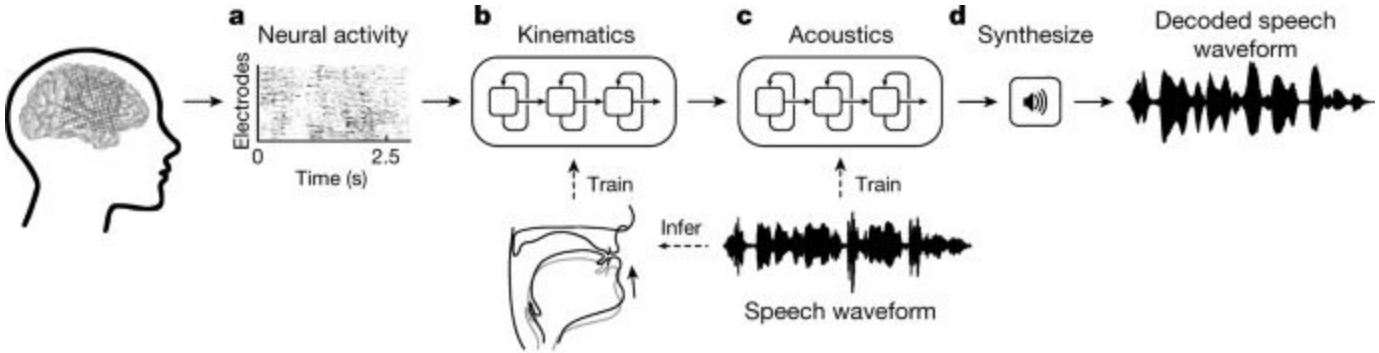
Jones et al. 2007

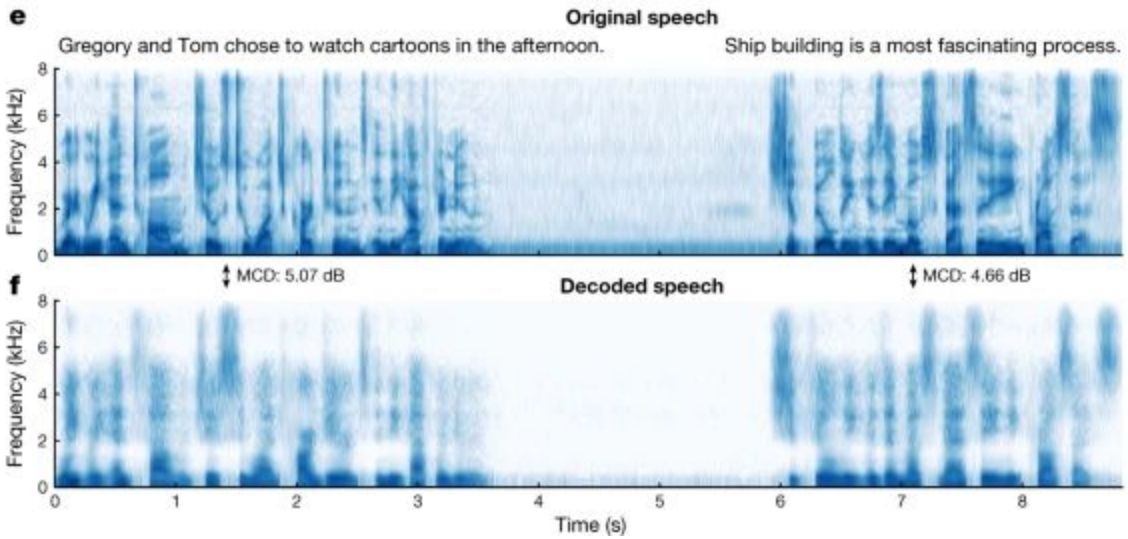
Motor Decoding



Pandarinath et al. 2018

Speech Decoding





Anumanchipalli et al. 2019

Decoding

Used in different animal models (human, monkey, rat, mice, etc)

Used for basic science and for brain computer interfaces

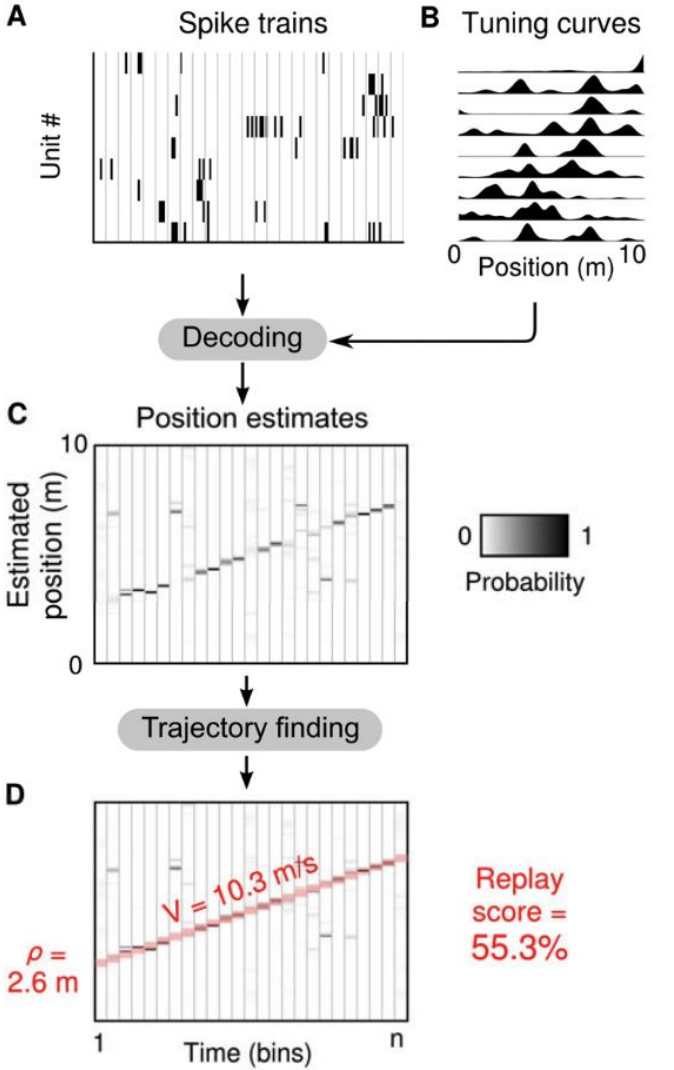
Used with many different brain areas

Used to decode everything from sensory to cognitive

Used to decode both discrete and continuous variables

We can’t cover everything here, but we are going to learn about most common decoding used in spatial neuroscience

Bayesian decoding



Notice this involves using the **encoding model**

This is a form of **supervised decoding**

Davidson et al. 2009

Review: Point process encoding models

We need to estimate the tuning curves over position. We can do this with:

- Occupancy Normalized Histograms
- Occupancy Normalized Kernel Density Estimation
- Poisson regression

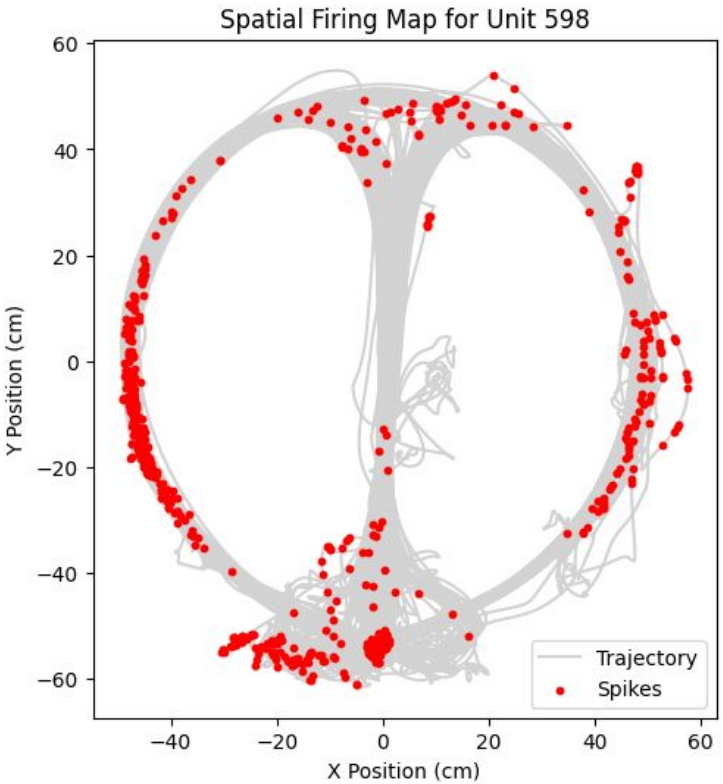
We assume (in this basic case) that the neurons are only driven by position. So we need to know how firing rate changes by position of the animal.

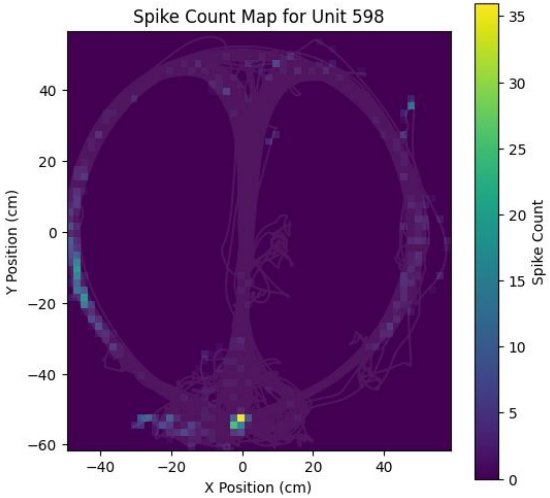
Rate maps (e.g. place fields)

We could take a histogram over position, but this would be just counts of spikes per position. The animal could be in one position longer.

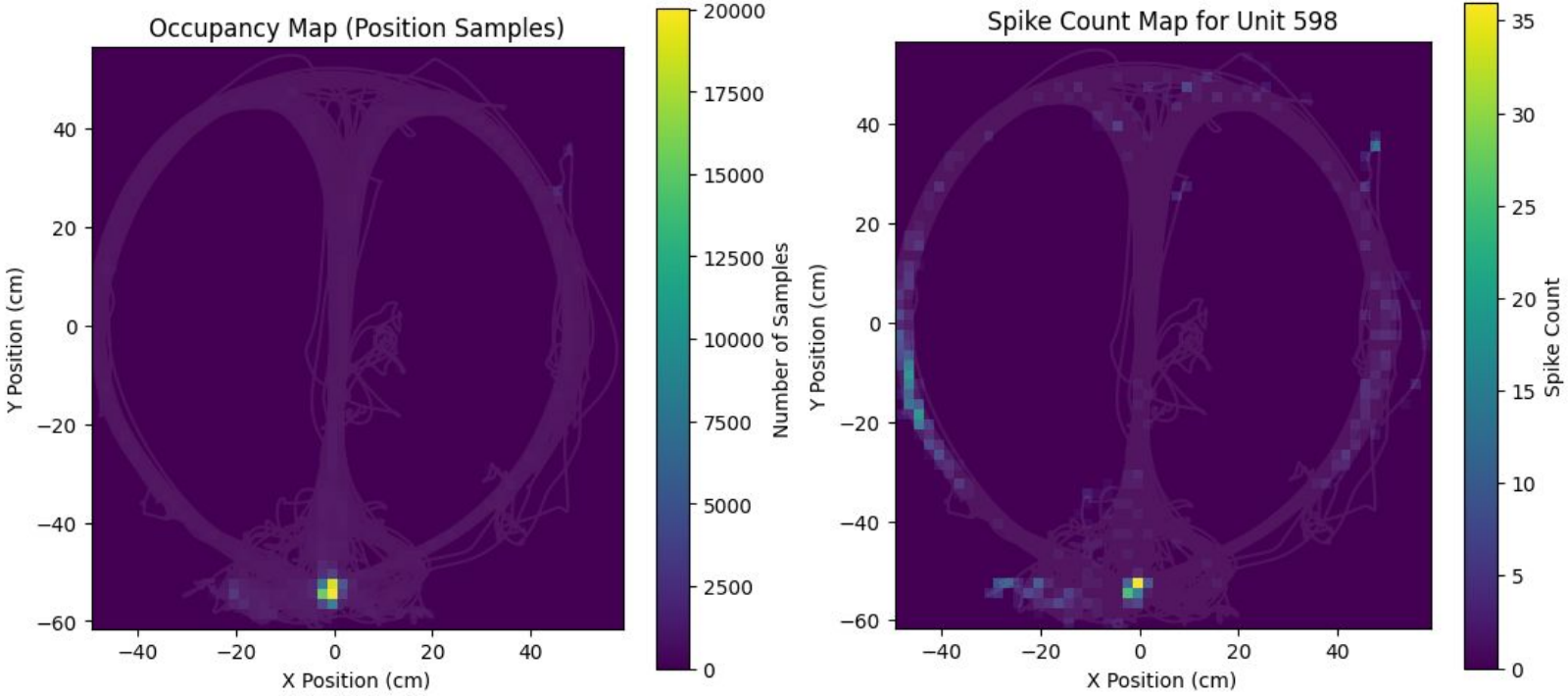
We want spike rate (spikes per second for each position bin)

How do we do this?



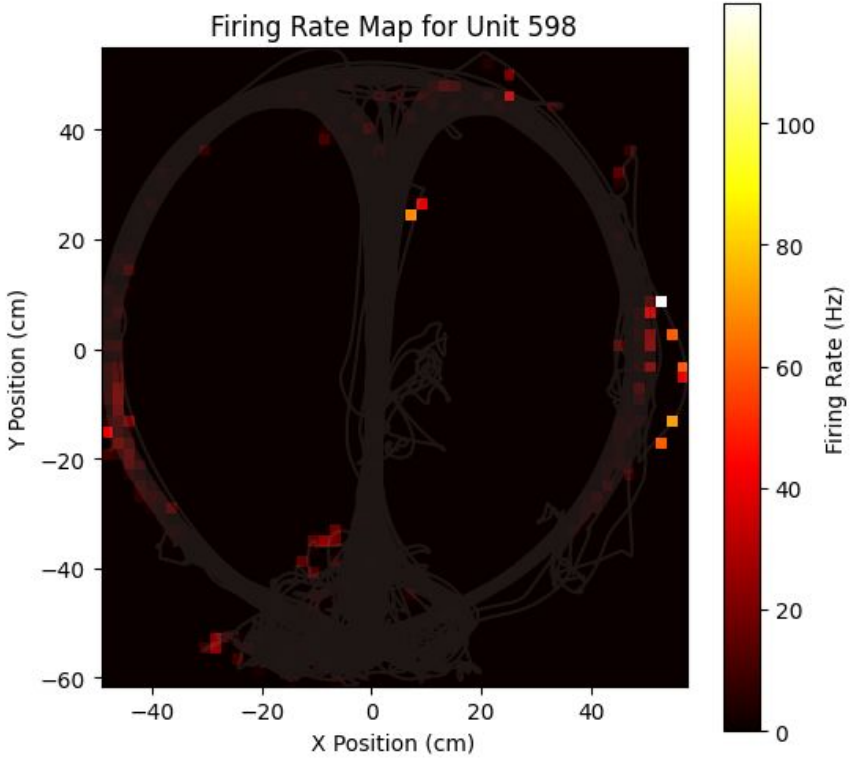


Occupancy - How often is the animal at a given position



Rate map - occupancy normalized histogram

Normalize counts per position bin by how often the animal is there.



Poisson distribution

Probability of k spikes for a given time bin:

$$P(k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

lambda = rate (expected number of events)

P(k) = probability of k spikes

What is the probability of 0 spikes?

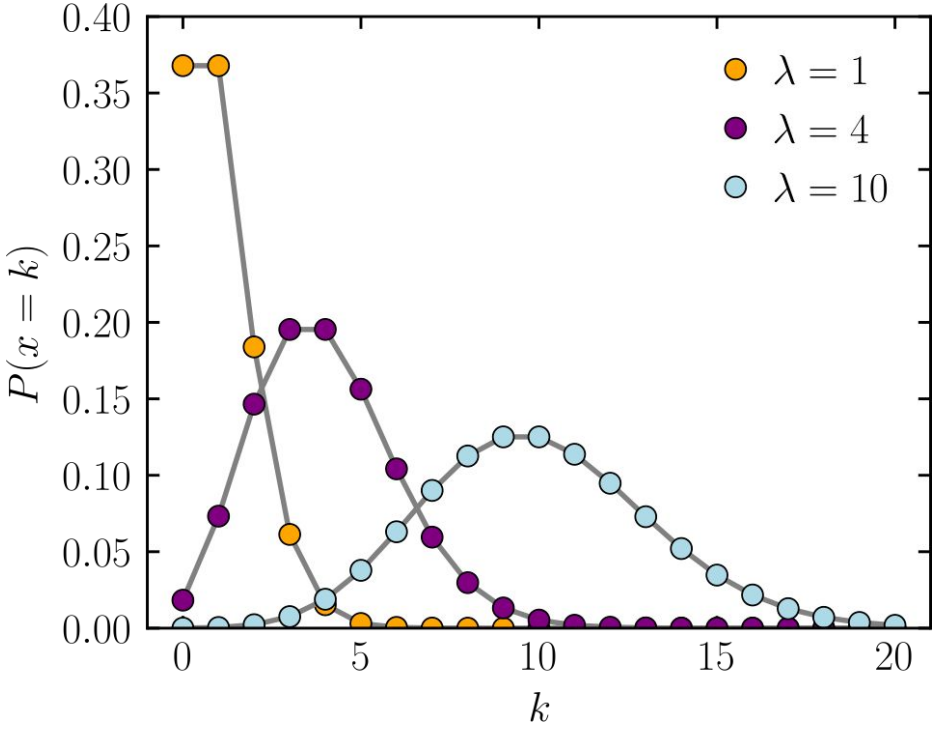
Poisson distribution

Low rate (lambda = 1): most probable one spike

Medium rate (lambda = 4):

most probable 4 spikes but long tail

High rate (lambda = 10): Gaussian-like



Review: Poisson likelihood measures how well a particular parameter setting explains the data, assuming the model is true.

In each time bin, the neuron generates spikes according to a Poisson distribution with mean rate lambda.

We can score how well each set of parameters fits the data. For each time bin, the Poisson probability is:

$$P(k_i \mid \lambda_i) = \frac{\lambda_i^{k_i} e^{-\lambda_i}}{k_i!}$$

Stats Background: Joint Probability

	Male	Female	Total
Football	0.24	0.15	0.39
Rugby	0.2	0.05	0.25
Other	0.1	0.26	0.36
	0.54	0.46	1

Stats Background: Joint Probability

	Male	Female	Total
Football	0.24	0.15	0.39
Rugby	0.2	0.05	0.25
Other	0.1	0.26	0.36
	0.54	0.46	1

Joint prob of sex and sport

Stats Background: Marginal Probability

	Male	Female	Total
Football	0.24	0.15	0.39
Rugby	0.2	0.05	0.25
Other	0.1	0.26	0.36
	0.54	0.46	1

Stats Background: Conditional Probability

Conditional probability is a measure of the probability of an event occurring, given that another event (by assumption, presumption, assertion, or evidence) is already known to have occurred.

$$P(A \mid B) = \frac{P(A, B)}{P(B)}$$

Stats Background: Conditional Probability

What is P(A)?

$P(A) = 0.30 + 0.10 + 0.12 = 0.52$

What is $P(A|B_1)$?

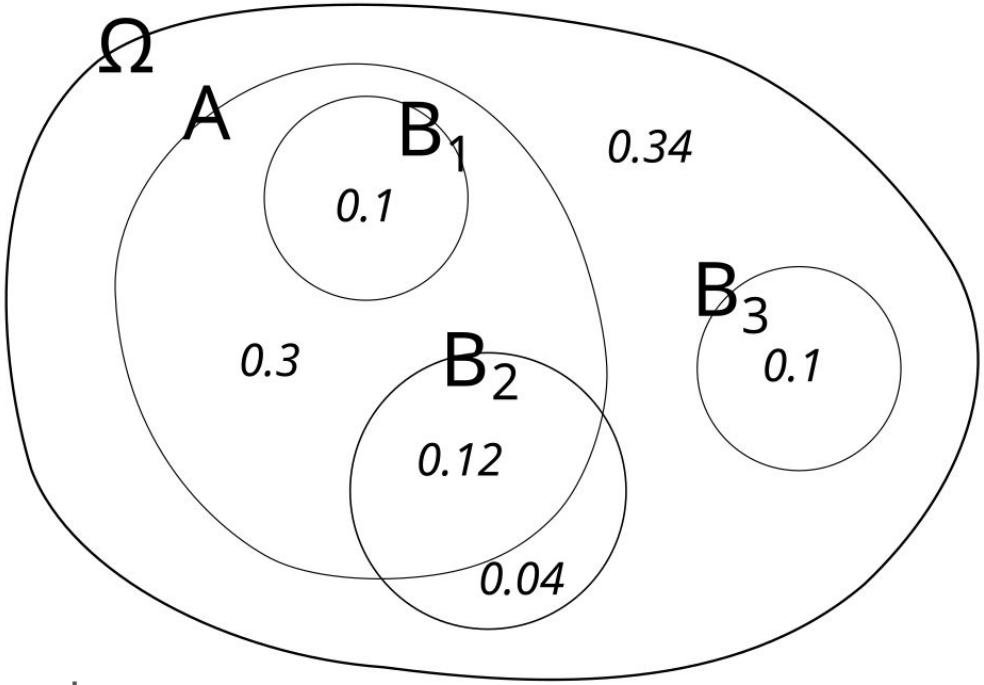
$P(A|B_1) = 1$

What is $P(A|B_2)$?

$P(A|B_2) = 0.12 \div (0.12 + 0.04) = 0.75$

What is $P(A|B_3)$?

$P(A|B_3) = 0$



Reduce the search space

Stats Background: Bayes Theorem

“reverse the conditional”

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Stats Background: Bayes Theorem

For decoding, we usually work with the proportional version and then end up normalizing by P(B) in the end

$$P(A \mid B) \propto P(B \mid A)\, P(A)$$

$$P(B) = \sum_{A} P(B \mid A) P(A)$$

Stats Background: Independence

Independence means the joint factorizes into the product of the marginals

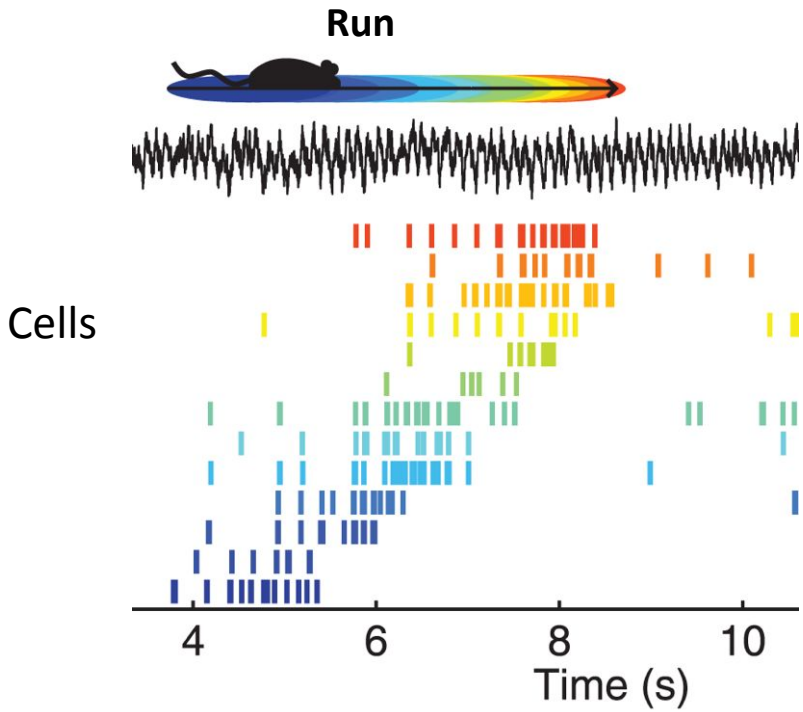
$$P(A, B) = P(A)\,P(B)$$

Stats Background: Conditional Independence

Conditional independence means that once you know C, learning B tells you nothing more about A

$$P(A, B \mid C) = P(A \mid B, C)\, P(B \mid C) = P(A \mid C)\, P(B \mid C)$$

Bayesian Decoder



Want to infer:

$$p\left(x_t \middle| \Delta N_{1:t}^{1:C}\right)$$

Probability of latent position x given spiking from C cells at time t

t	Time
$\Delta N^{1:C}$	Observed spikes from C cells
x	Latent position

Adapted from Diba & Buszaki 2007

What is a latent state?

A latent state is **not directly observable,** but must be inferred via mathematical model through other observations.

In this case, the latent state variable **x is the mental position of the animal.**

Our observations are the **spike times.**

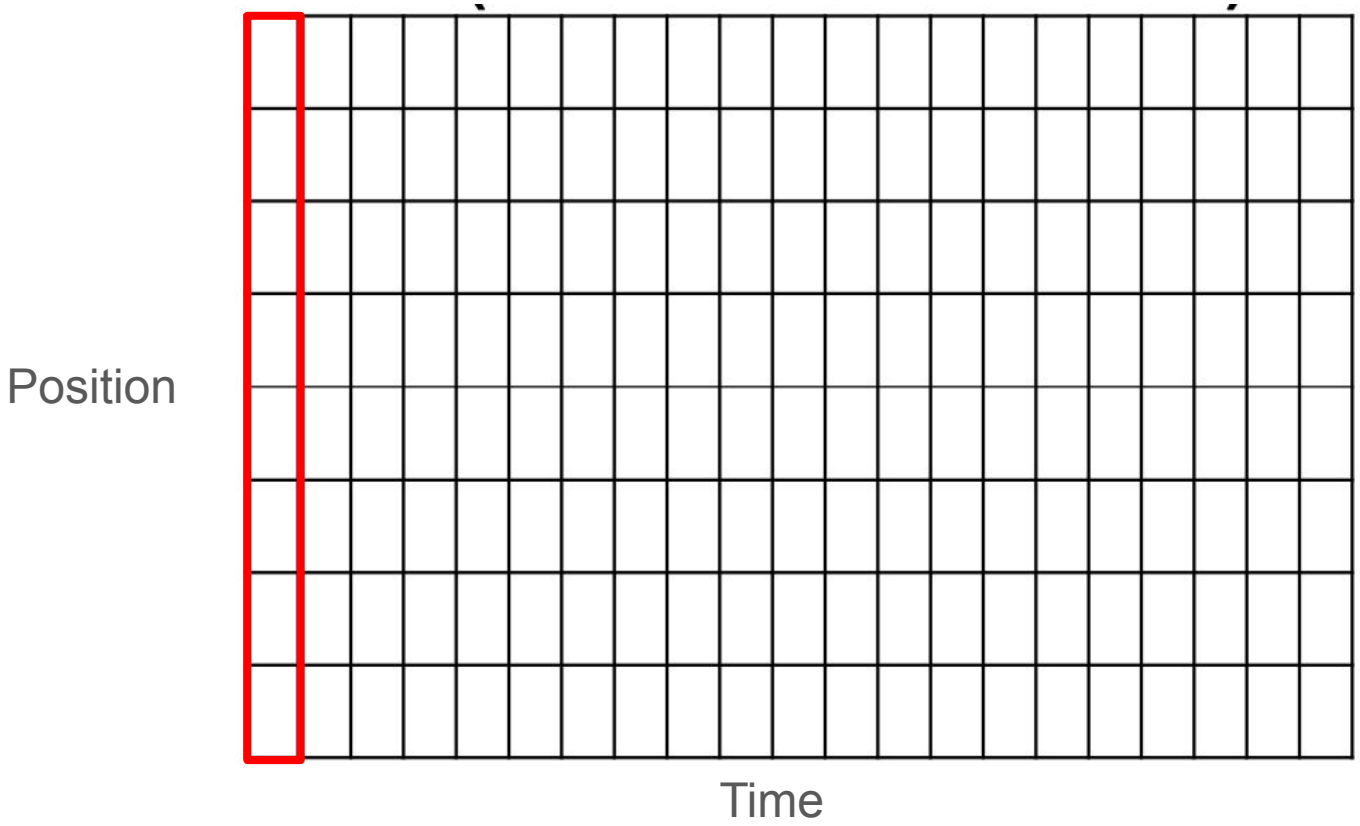
The **actual position of the animal** is observable.

Want to infer:

$$p\left(x_t \middle| \Delta N_{1:t}^{1:C}\right)$$

Probability of latent position x given spiking from C cells at time t

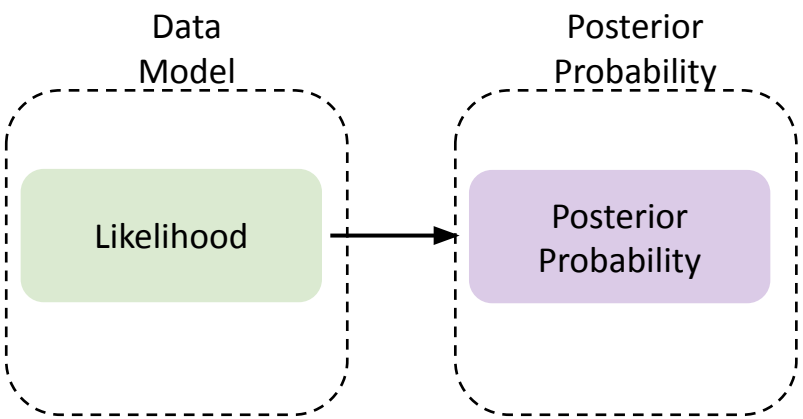
Bayesian Decoder



Subdivide time and position into bins

We will consider each time bin

Bayesian Decoder: for each time bin



$$p(x_t|\Delta N_{1:t}^{1:C}) \propto p(\Delta N_t^{1:C}|x_t) * p(x_t)$$

Probability of Position

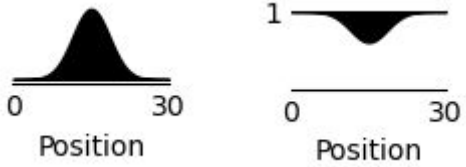
Poisson Likelihood

Prior Probability of Position

If spike

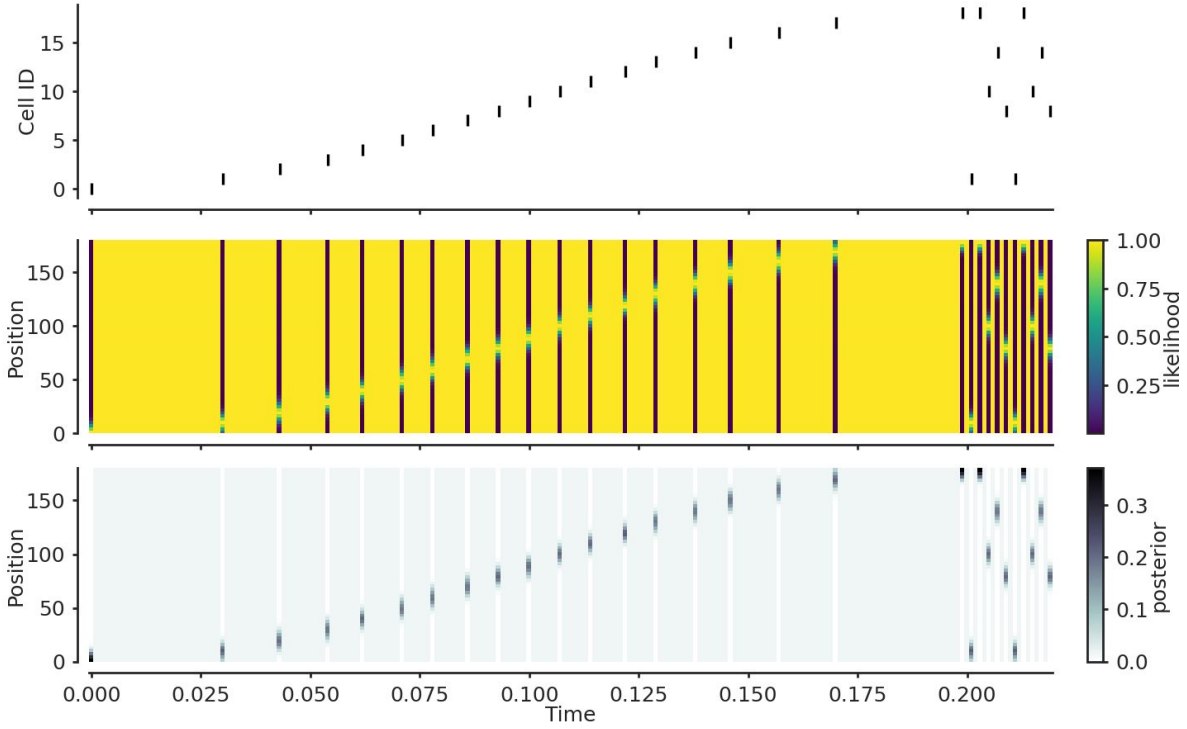
$$\prod_{i=1}^{C} [\lambda_i(t_k \mid x_k)\Delta_k]^{N_{t_k}^i} * e^{-\lambda_i(t_k \mid x_k)\Delta_k}$$

If no spike



t	Time
$\Delta N^{1:C}$	Observed spikes from C cells
x	Latent position

Example on simulated data



Likelihood

Posterior Probability

Practical issues: Numerical stability of likelihoods

Likelihoods tends to be small numbers (<1)

With many cells, we take the product of many small numbers

This can become too tiny for the computer to store, so everything looks like zero.

$$P(\{N_{t_k}^i\}_{i=1}^c \mid x_k) = \prod_{i=1}^{c} \left([\lambda_i(t_k \mid x_k)\Delta_k]^{N_{t_k}^i} \, e^{-\lambda_i(t_k|x_k)\Delta_k} \right)$$

Solution: Compute likelihood in log domain

Product becomes addition

$$\log P(\{N_{t_k}^i\}_{i=1}^c \mid x_k) = \sum_{i=1}^{c} \left[N_{t_k}^i \log\big(\lambda_i(t_k \mid x_k)\Delta_k\big) - \lambda_i(t_k \mid x_k)\Delta_k \right]$$

Solution: Compute decoder in log domain using stable softmax

$$\ell(x_t) \;=\; \log p(\Delta N_t^{1:C} \mid x_t) \;+\; \log p(x_t)$$

$$p(x_t \mid \Delta N_t^{1:C}) = \frac{\exp(\ell(x_t))}{\sum_{x_t'} \exp(\ell(x_t'))}$$

Normalizing with the log ends up being a special function called **the softmax**. But if the log posterior **L** becomes very large (either positive or negative), **the computer can still struggle to represent it accurately** after normalization.

Solution: Compute decoder in log domain using stable softmax

$$\ell(x_t) \;=\; \log p(\Delta N_t^{1:C} \mid x_t) \;+\; \log p(x_t)$$

$$m \;=\; \max_{x_t'} \ell(x_t')$$

$$p(x_t \mid \Delta N_t^{1:C}) = \frac{\exp(\ell(x_t) - m)}{\sum_{x_t'} \exp(\ell(x_t') - m)}$$

We can prevent this by subtracting the max log posterior value before normalization.

Practical issues: Choice of bin size

Need to choose time and position bin size.

Small time bins mean more bins with zero spikes. These might be minimally informative. Also less computational efficiency

Coarse time bins means blurs fast dynamics; assumes state/rate roughly constant within-bin

Small position bins mean more unoccupied bins or low sample bins / less computational efficiency

Coarse position bins means less spatial resolution

Practical Issues: Choice of smoothing in the encoding model rate maps

Too little smoothing: noisy fields → overconfident, spiky likelihoods

Too much smoothing: underfit → flattened likelihoods, poorer discrimination

This also interacts with the position bin size.

Practical Issues: Assumes we have the correct encoding model

Common mismatch: model uses only position, but spikes depend on more (speed, head direction, theta phase, reward, trajectory, etc., non-stationary place fields)

Unmodeled covariates → “extra variability” (over/under-dispersion) and biased posteriors

Practical issues: Assumes we have spike sorted correctly

This affects the estimate of the place rate map (encoding model)

It also affects decoding (which rate map is supposed to be active in a given time bin)

Practical Issues: Assumes we have the correct decoding model

If the brain variable is not strictly position (e.g., trajectory, goal, latent state), decoding only position can look noisy or wrong

Decoder may need state augmentation (position and direction) or multiple maps

Practical Issues: Assumes we have conditional independence of neurons

Shared input / correlations violate this → double-counting evidence → overconfident posteriors (too narrow)

Practical Issues: Computational efficiency

Cost scales with (#time bins)×(#position bins)×(#cells)

To mitigate, people often use 1D position (they project 2D position to 1D).

But then you have to consider how smoothing works in rate maps in 1D if the environment is complex

Practical Issues: Likelihood uses the expected (average) rate

Doesn’t propagate uncertainty in place-field location/shape → posteriors can be too confident, especially with limited data / drift

Practical Issues: High-rate neurons contribute more to the likelihood; informativeness depends on tuning, not just mean rate.

Firing rate does affect the likelihood. High-rate neurons (or bins with more spikes) contribute more evidence and can dominate the posterior.

What can be misleading is that baseline/high firing doesn't automatically mean "more informative"—informativeness depends on how strongly $\lambda(x)$ varies with x (tuning), not just its mean rate.

Practical Issues: Behavioral sampling bias / occupancy bias

Animals sample space unevenly → some position bins have **low occupancy**

In low-occupancy bins, the estimated rate $\lambda(x)$ is **high-variance / uncertain**

We plug in the **mean/point estimate** $\lambda(x)$ and ignore uncertainty

This can produce **spurious rate estimates** (too high *or* too low) in poorly sampled bins, which distorts the likelihood and yields **biased posteriors** (often favoring well-sampled regions, but can also create "attractor" bins if λ spikes)

Practical Issues: No-spike likelihood and incomplete field coverage (cell-sampling bias)

Many decoders treat silence as evidence via the Poisson term

$$\prod_{i=1}^{C} [\lambda_i(t_k \mid x_k)\Delta_k]^{N_{t_k}^i} * \boxed{e^{-\lambda_i(t_k \mid x_k)\Delta_k}}$$

If your recorded population doesn't "cover" the environment (few fields in some regions), then no spikes can incorrectly favor those under-covered locations (because predicted rates there are low for many cells)

Net effect: systematic bias toward regions with weak/absent tuning in the recorded set, especially in small Δt bins

Practical issues: Zero rates

Related to binning.

With many neurons, a single near-zero can dominate the sum.

If using the log likelihood, log of zero is infinity.

Practical issues

- Numerical stability of likelihoods
- Choice of bin size
- Choice of smoothing in the encoding model rate maps
- Assumes we have the correct encoding model
- Assumes we have spike sorted correctly
- Assumes we have the correct decoding model
- Assumes we have conditional independence of neurons
- Computational efficiency
- Likelihood uses the expected (average) rate
- High-rate neurons contribute more to the likelihood; informativeness depends on tuning, not just mean rate
- Behavioral sampling bias / occupancy bias
- No-spike likelihood and incomplete field coverage (cell-sampling bias)
- Zero rates
- Choice of prior

Priors

We haven't talked a lot about priors.

That's because everyone chooses to **make the prior Uniform**

Doesn't that make the probability of position the same as the Poisson likelihood? **Yes.**

$$p\left(x_t \middle| \Delta N_{1:t}^{1:C}\right) \propto p\left(\Delta N_t^{1:C} \middle| x_t\right) * p(x_t)$$

Probability of Position

Poisson Likelihood

Prior Probability of Position

When would you choose a prior?

A **prior** encodes some previous knowledge.

For example, if you believed the mental position of the animal was really likely to be in certain positions, you could use the prior to upweight those positions and downweight the other positions. This could **make the model more robust.**

A **uniform prior** encodes that we have no prior knowledge about the mental position of the animal.

How would you decode other variables?

Discrete variables (e.g. left vs. right turn)?

Circular variables (e.g. head direction)?

Bayesian decoder

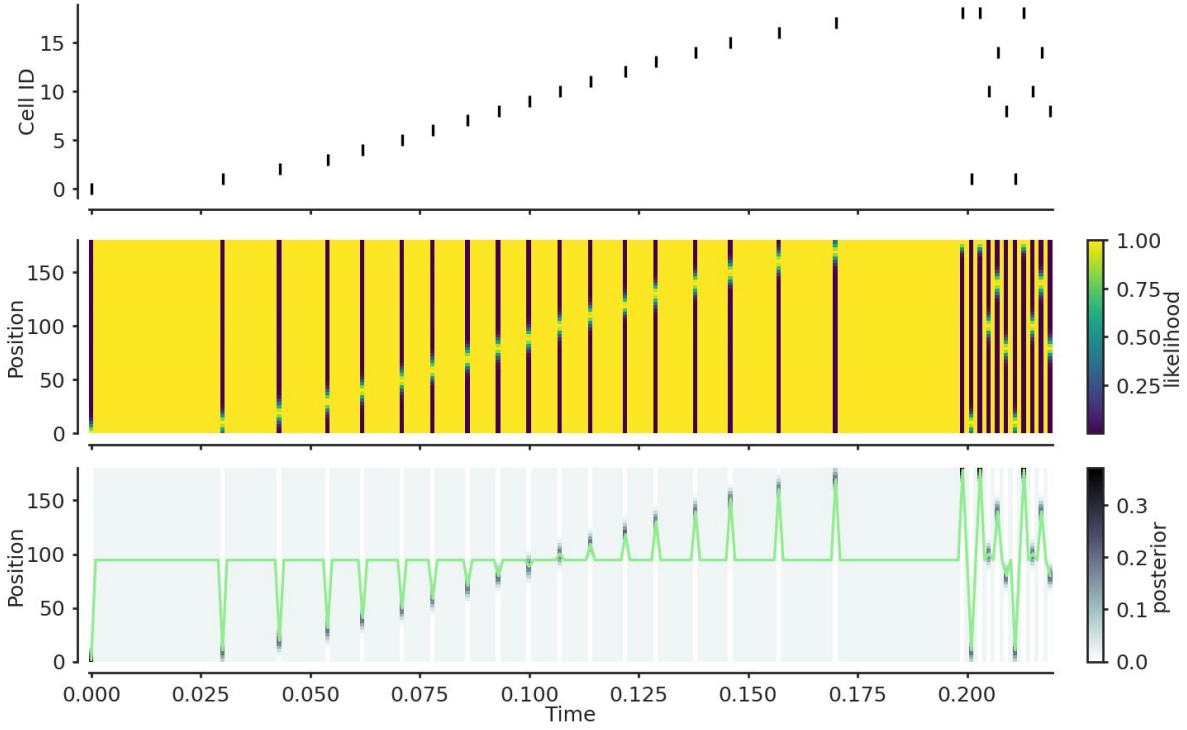
That's it, that's the Bayesian decoder:

$$p\left(x_t \middle| \Delta N_{1:t}^{1:C}\right) \propto p\left(\Delta N_t^{1:C} \middle| x_t\right) * p(x_t)$$

We now have probability of mental position over time for all possible positions.

But scientists often want a simpler summary. They want to find a single trajectory. How could we do that?

Maximum A Posteriori Estimate (MAP) finds the maximum probable position in each time bin.



Likelihood

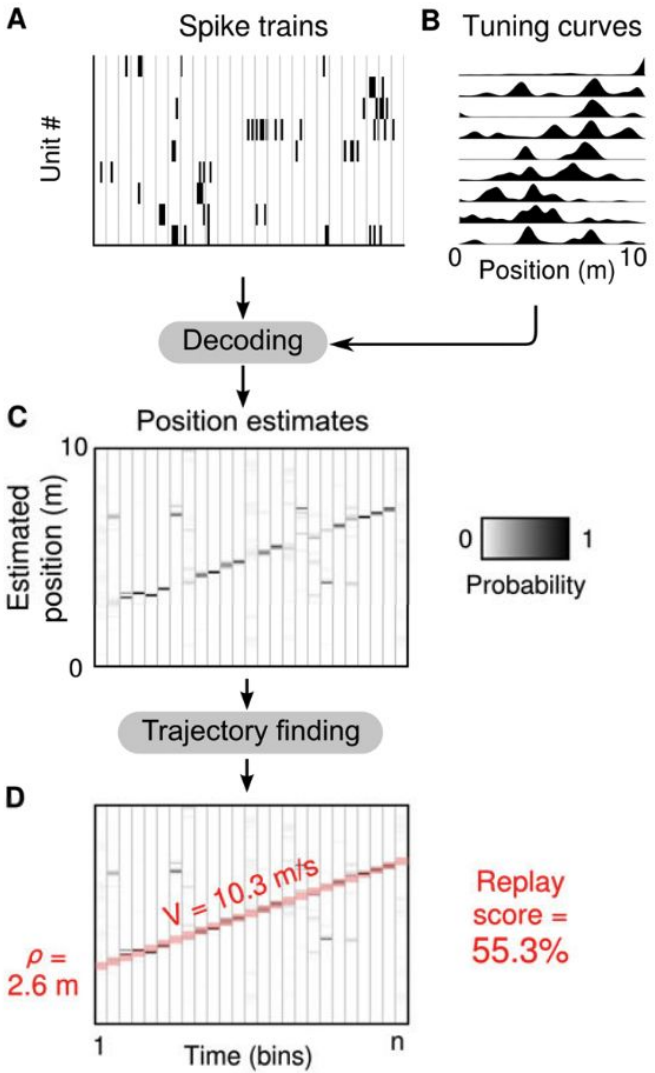
Posterior Probability

Most Probable Position

Issues with MAP estimate

- Noisy if small time bins:
  - Increase bin size (e.g., 20 ms) → more spikes → less jittery posterior
  - Cost: imposes a temporal scale; can't track fast changes within a bin
  - Also: binned-count Poisson assumes constant rate within the bin; larger bins make that assumption more fragile
- If the posterior is multi-modal, this will only pick one number.

We could fit a line to get a trajectory



Ways people do this

Radon transform

Weighted Correlation

Isotonic regression on MAP

Linear regression

Radon transform

Goal: Find the *single straight line* through the posterior image (time × position) that has the largest integrated probability mass.

How it works

- Treat the posterior matrix as an image.
- Compute a **Radon transform** (sinogram): integrate the image along many candidate line angles/offsets.
- Pick the line parameters (start position,velocity) corresponding to the **maximum** sinogram value.
- Optional: pre-smooth across nearby positions (convolution) to tolerate small spatial jitter; optional validity filter that removes lines that start/end outside track.

Output

- Start position (intercept), velocity (slope), the inferred line, and a normalized “score”.

Weighted correlation

Goal: Measure “how line-like” the posterior is via a **weighted correlation** between time and position.

How it works

- Computes correlation between time and position bins, weighted by posterior mass:

Output

- A single scalar correlation coefficient.

Isotonic regression on MAP

Goal: Fit a **monotonic** position-vs-time curve (nondecreasing or nonincreasing), using MAP as the observed trajectory but enforcing monotonicity.

How it works

- Compute per-time-bin MAP estimate
- Weight each point by the MAP probability (confidence).

Output

- A monotone predicted trajectory and a goodness-of-fit score.

Linear regression on posterior samples

Goal: Estimate a **best straight-line** trend a+b * t while accounting for posterior uncertainty by sampling.

How it works

- For each time bin, sample positions from the posterior
- Stack all samples across time and regress sampled xxx on time ttt using ordinary least squares.
- This approximates fitting a line to the posterior mass, not just the MAP path.

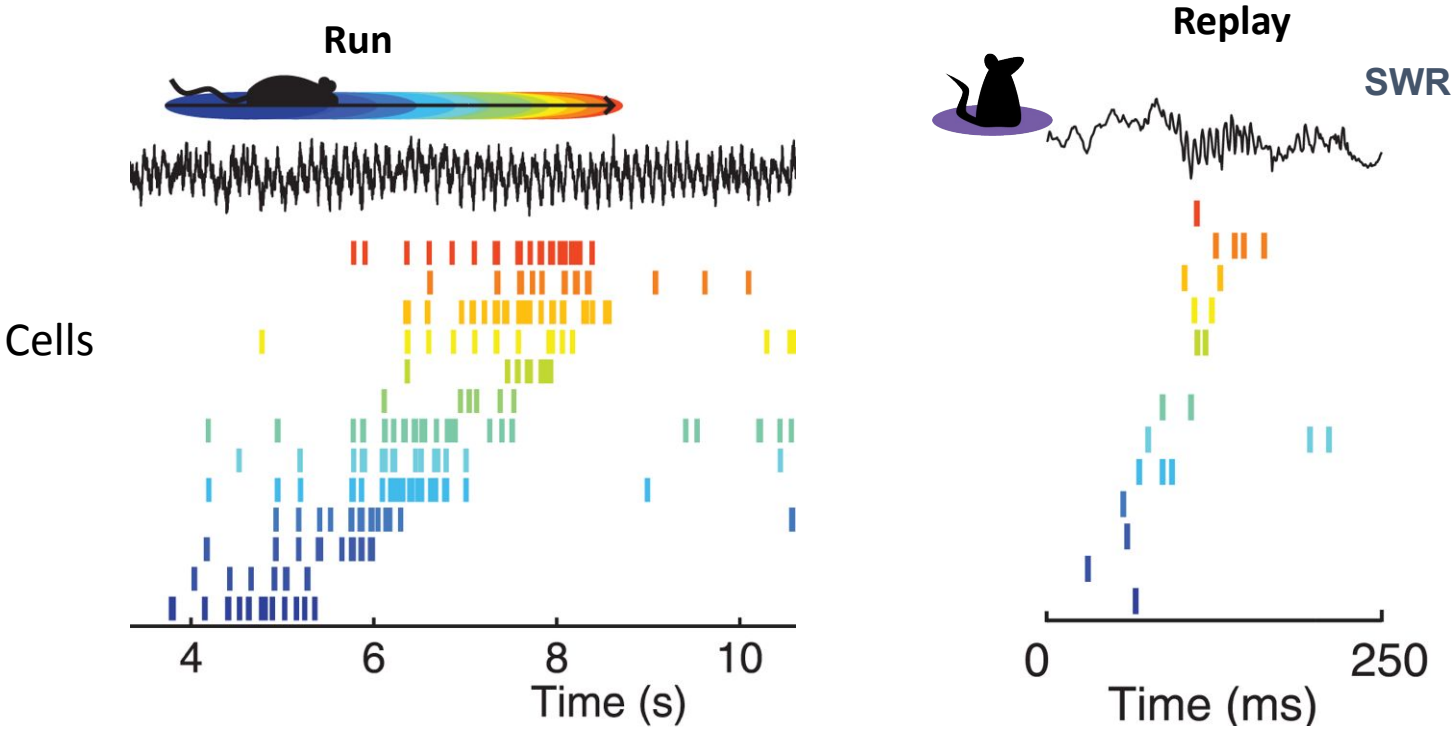
Output

- Intercept, slope, R^2, and predicted line.

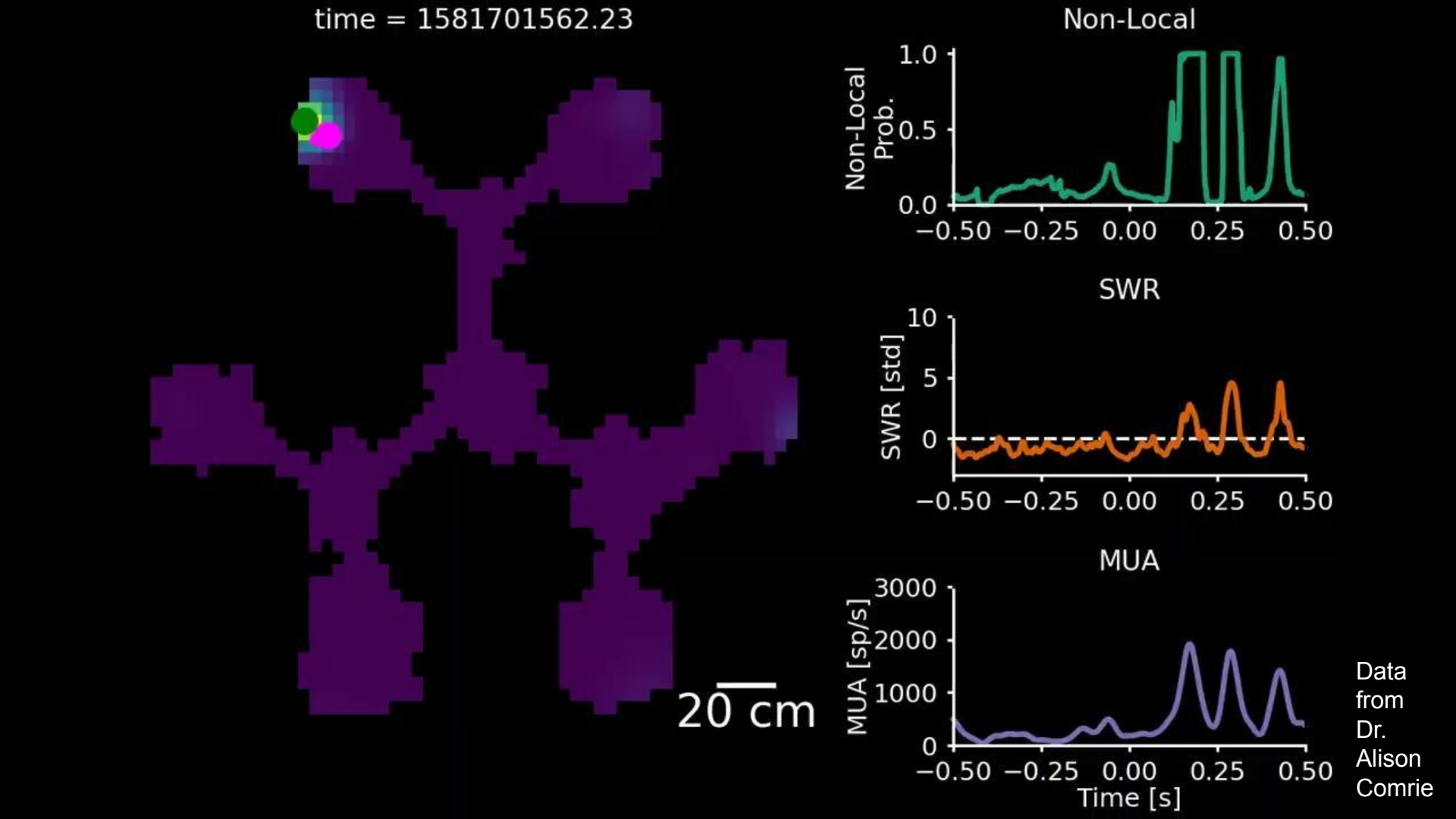
Issues with fitting lines

- All assume trajectories are lines.
  - May not accurately capture curved trajectories
  - This only works for 1D positions where there are no junctions (this will make adjacent positions not necessarily next to each other in 2D space)
- Impact of bin size and smoothing
  - The fits all strongly depend on the bin size and any smoothing (both of rate or between windows)
- Line fit is global
  - Choice of decoding window can affect the fit (adding time bins or subtracting bins on to the edges might change the fit)
- Does not handle multimodal posteriors

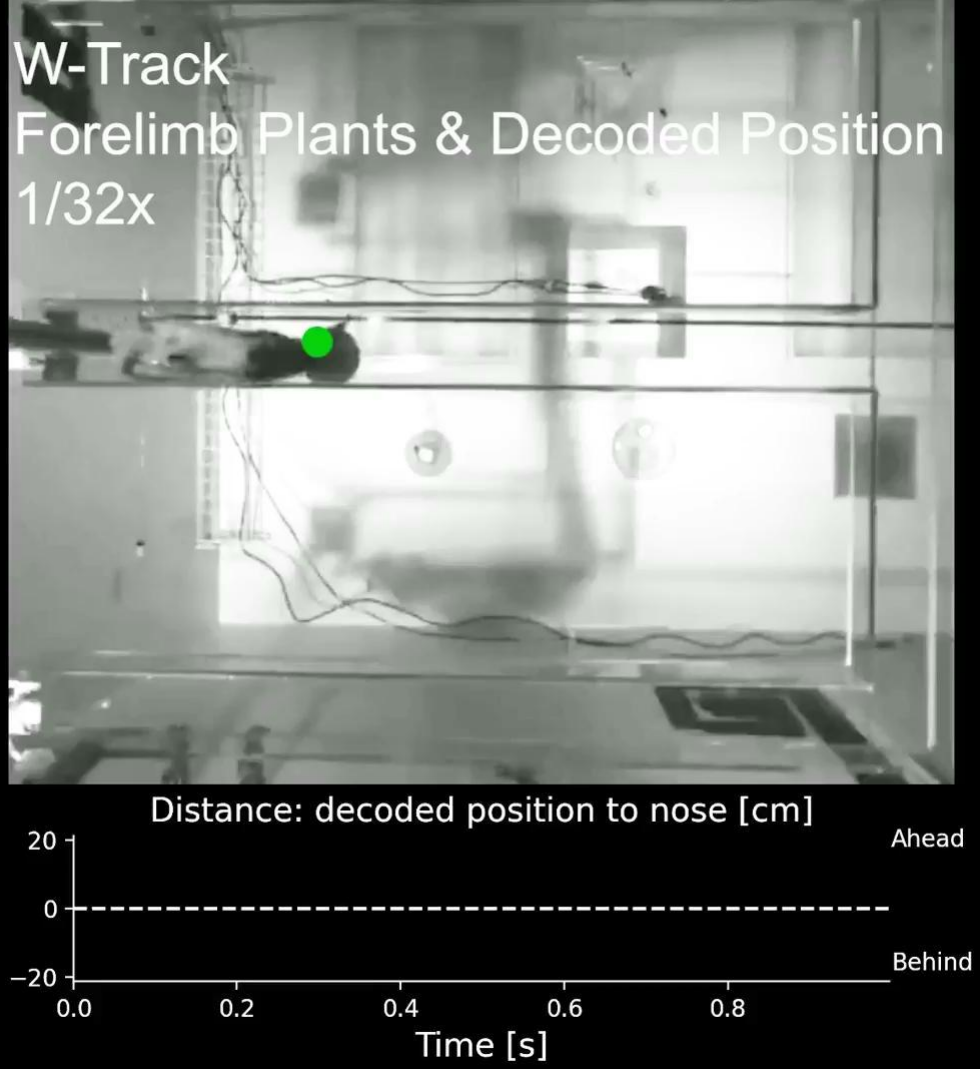
Hippocampal replay



Adapted from Diba & Buszaki 2007



Data from Dr. Alison Comrie



Joshi et al. 2023

Shuffles (Permutation tests) for significance testing

Question: “Is the decoded structure stronger than expected by chance given some null hypothesis?”

So the first step is: **what structure do you want to destroy, and what do you want to preserve?**

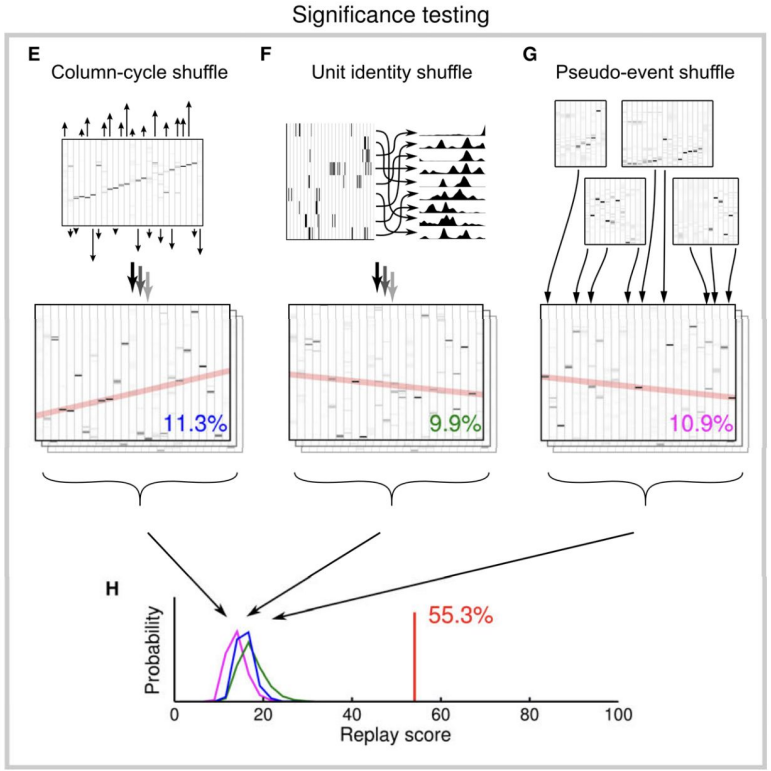
Common shuffle types (and what they preserve/destroy)

Shuffle spike times (per neuron)

Shuffle cell identity (permute neuron labels)

Shuffle place field locations (shift/permute tuning curves)

Pseudo-event shuffles (event surrogates)



Davidson et al. 2009

Shuffle spike times (per neuron)

How: jitter or circularly shift each neuron's spike train within the event/session.
Preserves: each neuron's spike count / ISI statistics (roughly), overall rate.
Destroys: alignment between spikes and the event's decoded trajectory; fine spike-time coordination across neurons.

Variants

- Circular shift within event (preserves within-event rate profile but breaks alignment to position)
- Jitter spikes by random offsets (preserves rate, breaks precise timing)

Pitfalls: can unintentionally destroy true non-Poisson structure (bursts/refractory), and may be too conservative or not conservative depending on what you preserve.

Shuffle cell identity (permute neuron labels)

How: permute cell IDs when computing likelihood / mapping spikes to tuning curves.
Preserves: population spike count statistics within the event.

Destroys: the mapping between a neuron's spikes and its own place field.

Use when: you want a null that keeps "how many spikes happened" but breaks spatial tuning alignment.

Pitfalls: if there are strong rate differences across cells, shuffling IDs changes the effective evidence structure in a way that's not purely "break spatial coding."

Shuffle place field locations (shift/permute tuning curves)

How: circularly shift each neuron's place field along position bins, or permute spatial bins in the tuning curve.
Preserves: each neuron's tuning *shape* and rate distribution.
Destroys: correct alignment of field location to the real track.

Use when: you want to preserve tuning smoothness but remove true spatial map.

Pitfalls: on tracks with discontinuities/junctions, "circular shifts" can create impossible fields (teleporting across branches). Also interacts with masking/interior bins.

Pseudo-event shuffles (event surrogates)

How: draw events from non-event periods matched for duration, spike count, speed/state, ripple power, etc.
Preserves: overall recording statistics and state-dependent firing.
Destroys: coupling specific to the detected events.

Use when: the concern is “events are special” (e.g., ripples, immobility) and you want a matched null.

Pitfalls: matching is hard—if you don’t match key covariates (speed/theta/ripple amplitude), you test state differences rather than replay structure.

Problems with shuffles

Null mismatch: a shuffle may destroy more (or less) structure than intended → p-values don't answer the question you think.

Dependence and circularity: using the same data to detect events, tune hyperparameters, and test significance can inflate significance.

Multiple comparisons: many events, many windows, many statistics → need correction or hierarchical testing.

Preserving spike count vs. preserving correlations: different shuffles keep different second-order structure (shared gain, synchrony). Your significance can flip depending on which null you pick.

Too few shuffles: limited B gives coarse p-values; the minimum achievable is 1/(B+1)

Choice of statistic matters: line-based scores can be sensitive to binning/smoothing; shuffles inherit that sensitivity.

Summary

The Bayesian decoder is conceptually simple: each neuron contributes evidence in each time bin, combined with a prior to form a posterior over position.

In practice, many choices strongly affect the result (binning, smoothing, model assumptions, and data quality).

How you summarize the posterior (MAP, line fits) can change the inferred trajectory.

Significance testing is also nontrivial: conclusions depend on the chosen null hypothesis and shuffle/permutation scheme.

Next time…

Ways we can improve upon the basic Bayesian Decoder